

0328llm_task2_2_Llama32_1B_buyer_agentic_workflo

March 28, 2026

0.1 Buyer agentic workflow pipeline

Research Q4b: How can agentic workflows for sellers (e.g., listing optimization, price feedback) and buyers (e.g., preference matching, personalized recommendations) be designed using LLMs?

```
[3]: # 1. Imports and LLM configuration
from typing import Dict, Any # <- needed for type hints

import os
import pandas as pd
import numpy as np
import joblib
import requests
import textwrap

LM_STUDIO_URL = "http://localhost:1234/v1/chat/completions"
LM_MODEL_NAME = "llama-3.2-1b-instruct" # matches the API identifier in LM
↳Studio

# 2. Load the enhanced dataset (with sentiment scores)
current_dir = os.path.dirname(os.path.abspath("__file__")) if "__file__" in_
↳globals() else os.getcwd()
output_dir = os.path.join(current_dir, "output")

data_file = os.path.join(output_dir, "df_with_extracted_scores.csv")
df = pd.read_csv(data_file)
print("Loaded data:", data_file)
print("Columns:", df.columns.tolist())

# 3. LLM helper for buyer assistant (reuses the same API as seller notebook)
def call_llm(prompt: str) -> str:
    """
    Call LLaMA-3.2-1B-Instruct via LM Studio's local OpenAI-compatible API.
    Make sure LM Studio server is running (Status: Running).
    """
    payload = {
```

```

        "model": LM_MODEL_NAME,
        "messages": [
            {"role": "system", "content": "You are a helpful real-estate_
↪assistant."},
            {"role": "user", "content": prompt},
        ],
        "temperature": 0.3,
        "max_tokens": 512,
    }

    try:
        resp = requests.post(LM_STUDIO_URL, json=payload, timeout=120)
        resp.raise_for_status()
        data = resp.json()
        return data["choices"][0]["message"]["content"]
    except Exception as e:
        print("Error calling LLM:", e)
        return "Error: LLM call failed."

# 4. Buyer preference structure (input config)
def default_buyer_preferences() -> Dict[str, Any]:
    """
    Example buyer preferences for testing.
    """
    return {
        # Budget wide enough to include ~£25M luxury properties
        "budget_min": 20_000_000,
        "budget_max": 30_000_000,

        # Match large luxury family homes
        "min_bedrooms": 6,
        "min_bathrooms": 6,
        "min_size_sqft": 10_000,

        "preferred_property_types": ["House"],

        # Emphasise luxury and good transport/schools
        "weight_luxury": 0.5,
        "weight_transport": 0.25,
        "weight_school": 0.25,
        "weight_budget_closeness": 0.2,

        # Show a few top candidates around that segment
        "top_k": 5,
    }

```

```

# 5. Filter + ranking for buyer preferences (data-side logic)
def filter_properties_for_buyer(df: pd.DataFrame,
                                prefs: Dict[str, Any]) -> pd.DataFrame:
    """
    Filter the full listings DataFrame according to buyer preferences.
    Assumes df has columns: price, bedrooms, bathrooms, sizeSqFeetMax,
    propertyType, luxury_score, transport_score, school_score, renovation_score.
    """
    filtered = df.copy()

    # Numeric filters
    if "budget_min" in prefs:
        filtered = filtered[filtered["price"] >= prefs["budget_min"]]
    if "budget_max" in prefs:
        filtered = filtered[filtered["price"] <= prefs["budget_max"]]

    if "min_bedrooms" in prefs:
        filtered = filtered[filtered["bedrooms"] >= prefs["min_bedrooms"]]
    if "min_bathrooms" in prefs:
        filtered = filtered[filtered["bathrooms"] >= prefs["min_bathrooms"]]
    if "min_size_sqft" in prefs:
        filtered = filtered[filtered["sizeSqFeetMax"] >= prefs["min_size_sqft"]]

    # Categorical filter: propertyType
    if prefs.get("preferred_property_types"):
        filtered = filtered[filtered["propertyType"].
        ↪isin(prefs["preferred_property_types"])]

    return filtered

def add_matching_score(filtered: pd.DataFrame,
                        prefs: Dict[str, Any]) -> pd.DataFrame:
    """
    Add a 'matching_score' column representing how well each property
    matches the buyer's preferences (simple heuristic scoring).
    """
    df_scored = filtered.copy()

    # Normalised sentiment contributions (0-1)
    df_scored["luxury_match"] = df_scored["luxury_score"] / 2.0 # 0-2 -> 0-1
    df_scored["transport_match"] = df_scored["transport_score"] # 0 or 1
    df_scored["school_match"] = df_scored["school_score"] # 0 or 1

    w_lux = prefs.get("weight_luxury", 0.4)
    w_tr = prefs.get("weight_transport", 0.3)
    w_sc = prefs.get("weight_school", 0.3)

```

```

sentiment_score = (
    w_lux * df_scored["luxury_match"]
    + w_tr * df_scored["transport_match"]
    + w_sc * df_scored["school_match"]
)

# Budget closeness: 1.0 if near the middle of the budget range
mid_budget = 0.5 * (prefs["budget_min"] + prefs["budget_max"])
max_dev = 0.5 * (prefs["budget_max"] - prefs["budget_min"]) + 1e-9
budget_dev = (df_scored["price"] - mid_budget).abs() / max_dev
budget_closeness = 1.0 - budget_dev.clip(0, 1)

w_budget = prefs.get("weight_budget_closeness", 0.2)

df_scored["matching_score"] = sentiment_score + w_budget * budget_closeness

return df_scored

# 6. Build LLM prompt for buyer recommendations
def build_buyer_prompt(prefs: Dict[str, Any],
                      candidates: pd.DataFrame) -> str:
    """
    Build a compact text prompt describing buyer preferences and
    top candidate properties for LLaMA-3.2-1B-Instruct.
    """
    # Limit number of candidates to keep prompt short
    top_k = prefs.get("top_k", 5)
    subset = candidates.head(top_k).copy()

    rows_text = []
    for idx, row in subset.reset_index(drop=True).iterrows():
        pid = f"P{idx+1}" # simple property ID
        rows_text.append(
            f"{pid}: price=${row['price']:, .0f}, "
            f"type={row['propertyType']}, "
            f"size={row['sizeSqFeetMax']} sqft, "
            f"beds={row['bedrooms']}, baths={row['bathrooms']}, "
            f"luxury_score={row['luxury_score']}, "
            f"transport_score={row['transport_score']}, "
            f"school_score={row['school_score']}, "
            f"renovation_score={row['renovation_score']}"
        )

    properties_block = "\n".join(rows_text) if rows_text else "No properties_
    found."

```

```

    prefs_text = f"""
Budget range: £{prefs['budget_min']:, .0f} - £{prefs['budget_max']:, .0f}
Minimum bedrooms: {prefs['min_bedrooms']}
Minimum bathrooms: {prefs['min_bathrooms']}
Minimum size: {prefs['min_size_sqft']} sqft
Preferred property types: {", ".join(prefs['preferred_property_types'])}
"""

    instructions = f"""
You are an assistant helping a home buyer choose suitable properties.

Buyer preferences:
{prefs_text.strip()}

Candidate properties (each with an ID):
{properties_block}

Task:
- You MUST ONLY refer to properties by the IDs listed above (P1, P2, P3, etc.).
  Never invent new IDs or mention properties like P5, P6, etc. if they do not
  appear in the list.
- If there are no suitable properties, explain this politely and suggest how
  the buyer could adjust their criteria.
- Otherwise:
  1) Identify the single best matching property ID from the list.
  2) Suggest 1-2 additional alternatives from the SAME list.
  3) Explain in a short paragraph why these properties fit the buyer's needs,
     focusing on budget, size, bedrooms, and the sentiment scores
     (luxury_score, transport_score, school_score, renovation_score).
- Keep the style clear and conversational.
"""

    return instructions.strip()

def recommend_properties_with_llm(prefs: Dict[str, Any],
                                  candidates: pd.DataFrame) -> str:
    """
    Wrapper that builds the buyer prompt and sends it to the LLM.
    """
    prompt = build_buyer_prompt(prefs, candidates)
    return call_llm(prompt)

# 7. End-to-end buyer agent function (agentic workflow)
def buyer_assistant(

```

```

    buyer_prefs: Dict[str, Any]
) -> Dict[str, Any]:
    """
    High-level agentic workflow for a buyer.

    Steps:
    1) Filter the full dataset according to buyer preferences.
    2) Compute a matching_score for each remaining property.
    3) Sort by matching_score and select top-k candidates.
    4) Ask the LLM to recommend and explain best matches.

    Returns:
    - filtered_ranked: DataFrame of candidate properties with matching_score.
    - llm_buyer_advice: string with the LLM's recommendations/explanations.
    """
    # 1) Filter
    filtered = filter_properties_for_buyer(df, buyer_prefs)

    if filtered.empty:
        # Still ask LLM to explain "no matches", for consistency
        empty_df = filtered.copy()
        llm_output = recommend_properties_with_llm(buyer_prefs, empty_df)
        return {
            "filtered_ranked": empty_df,
            "llm_buyer_advice": llm_output,
        }

    # 2) Add matching score
    scored = add_matching_score(filtered, buyer_prefs)

    # 3) Sort by matching_score (descending)
    scored_sorted = scored.sort_values(by="matching_score", ascending=False)

    # 4) Ask LLM
    llm_output = recommend_properties_with_llm(buyer_prefs, scored_sorted)

    return {
        "filtered_ranked": scored_sorted,
        "llm_buyer_advice": llm_output,
    }

```

Loaded data:

```

c:\Users\Admin\Python\S8_Thesis_1\llm\output\df_with_extracted_scores.csv
Columns: ['title', 'propertyType', 'sizeSqFeetMax', 'bedrooms', 'bathrooms',
'listingUpdateReason', 'price', 'Date', 'listingDescription', 'luxury_score',
'transport_score', 'school_score', 'renovation_score']

```

```
[4]: # 8. Example usage (UI-style display for buyer, for thesis screenshots)
def display_buyer_session(buyer_prefs: Dict[str, Any], result: Dict[str, Any]):
    print("=" * 80)
    print("BUYER ASSISTANT - RECOMMENDATIONS".center(80))
    print("=" * 80)

    print("\n[Buyer preferences]")
    print(f"    Budget range           : &{buyer_prefs['budget_min']:, .0f} - "
          f"&{buyer_prefs['budget_max']:, .0f}")
    print(f"    Min bedrooms           : {buyer_prefs['min_bedrooms']}")
    print(f"    Min bathrooms          : {buyer_prefs['min_bathrooms']}")
    print(f"    Min size (sq ft)       : {buyer_prefs['min_size_sqft']}")
    print(
        "    Preferred property types: "
        f"{', '.join(buyer_prefs['preferred_property_types'])}"
    )

    print("\n" + "-" * 80)
    print("TOP MATCHING PROPERTIES".center(80))
    print("-" * 80)

    candidates = result["filtered_ranked"].head(buyer_prefs.get("top_k", 5))
    if candidates.empty:
        print("No properties found that satisfy these filters.")
    else:
        for i, (_, row) in enumerate(candidates.iterrows(), start=1):
            print(f"\n[Property P{i}]")
            print(f"    Price           : &{row['price']:, .0f}")
            print(f"    Type            : {row['propertyType']}")
            print(f"    Size (sq ft)    : {row['sizeSqFeetMax']}")
            print(f"    Bedrooms        : {row['bedrooms']}")
            print(f"    Bathrooms       : {row['bathrooms']}")
            print(f"    luxury_score    : {row['luxury_score']}")
            print(f"    transport_score : {row['transport_score']}")
            print(f"    school_score    : {row['school_score']}")
            print(f"    renovation_score: {row['renovation_score']}")
            print(f"    matching_score  : {row['matching_score']:.3f}")

        print("\n" + "-" * 80)
        print("LLM RECOMMENDATION SUMMARY".center(80))
        print("-" * 80)

        advice = result["llm_buyer_advice"]
        print(textwrap.fill(advice, width=80))

    print("\n" + "=" * 80)
```

```
# 9. Run one example buyer session
example_buyer_prefs = default_buyer_preferences()
buyer_result = buyer_assistant(example_buyer_prefs)
display_buyer_session(example_buyer_prefs, buyer_result)
```

=====

BUYER ASSISTANT - RECOMMENDATIONS

=====

[Buyer preferences]

Budget range : £20,000,000 - £30,000,000
Min bedrooms : 6
Min bathrooms : 6
Min size (sq ft) : 10000
Preferred property types: House

TOP MATCHING PROPERTIES

[Property P1]

Price : £24,950,000
Type : House
Size (sq ft) : 16749.0
Bedrooms : 8.0
Bathrooms : 8.0
luxury_score : 0
transport_score : 2
school_score : 2
renovation_score: 1
matching_score : 1.198

[Property P2]

Price : £24,950,000
Type : House
Size (sq ft) : 16749.0
Bedrooms : 8.0
Bathrooms : 8.0
luxury_score : 0
transport_score : 0
school_score : 2
renovation_score: 0
matching_score : 0.698

[Property P3]

Price : £29,950,000
Type : House

Size (sq ft) : 15608.0
Bedrooms : 6.0
Bathrooms : 6.0
luxury_score : 0
transport_score : 1
school_score : 0
renovation_score: 0
matching_score : 0.252

[Property P4]

Price : £29,500,000
Type : House
Size (sq ft) : 12960.0
Bedrooms : 7.0
Bathrooms : 7.0
luxury_score : 0
transport_score : 0
school_score : 0
renovation_score: 0
matching_score : 0.020

[Property P5]

Price : £29,500,000
Type : House
Size (sq ft) : 12960.0
Bedrooms : 7.0
Bathrooms : 7.0
luxury_score : 0
transport_score : 0
school_score : 0
renovation_score: 0
matching_score : 0.020

LLM RECOMMENDATION SUMMARY

I'd be happy to help you find your dream home. First, let me review the buyer's criteria: they're looking for a house with a price range of £20,000,000 - £30,000,000, minimum of 6 bedrooms, 6 bathrooms, and a size of at least 10,000 sqft. They also prefer luxury properties with high sentiment scores. Let me start by identifying the single best matching property ID from the list: P1 has all these requirements! It's priced at £24,950,000, has 8 bedrooms, 8 bathrooms, and a size of 16,749 sqft. The sentiment score is 0 for luxury, transport score is 2, school score is 2, and renovation score is 1. Here are two additional alternatives that fit the buyer's needs: * P3: £29,950,000, has 6 bedrooms, 6 bathrooms, and a size of 15,608 sqft. The sentiment scores are 0 for luxury, transport score is 1, school score is 0, and renovation score is 0. * P4: £29,500,000, has 7 bedrooms, 7 bathrooms, and a size of 12,960 sqft. The

sentiment scores are 0 for luxury, transport score is 0, school score is 0, and renovation score is 0. All three properties fit the buyer's needs, but P1 stands out due to its high price point and luxurious features.

=====